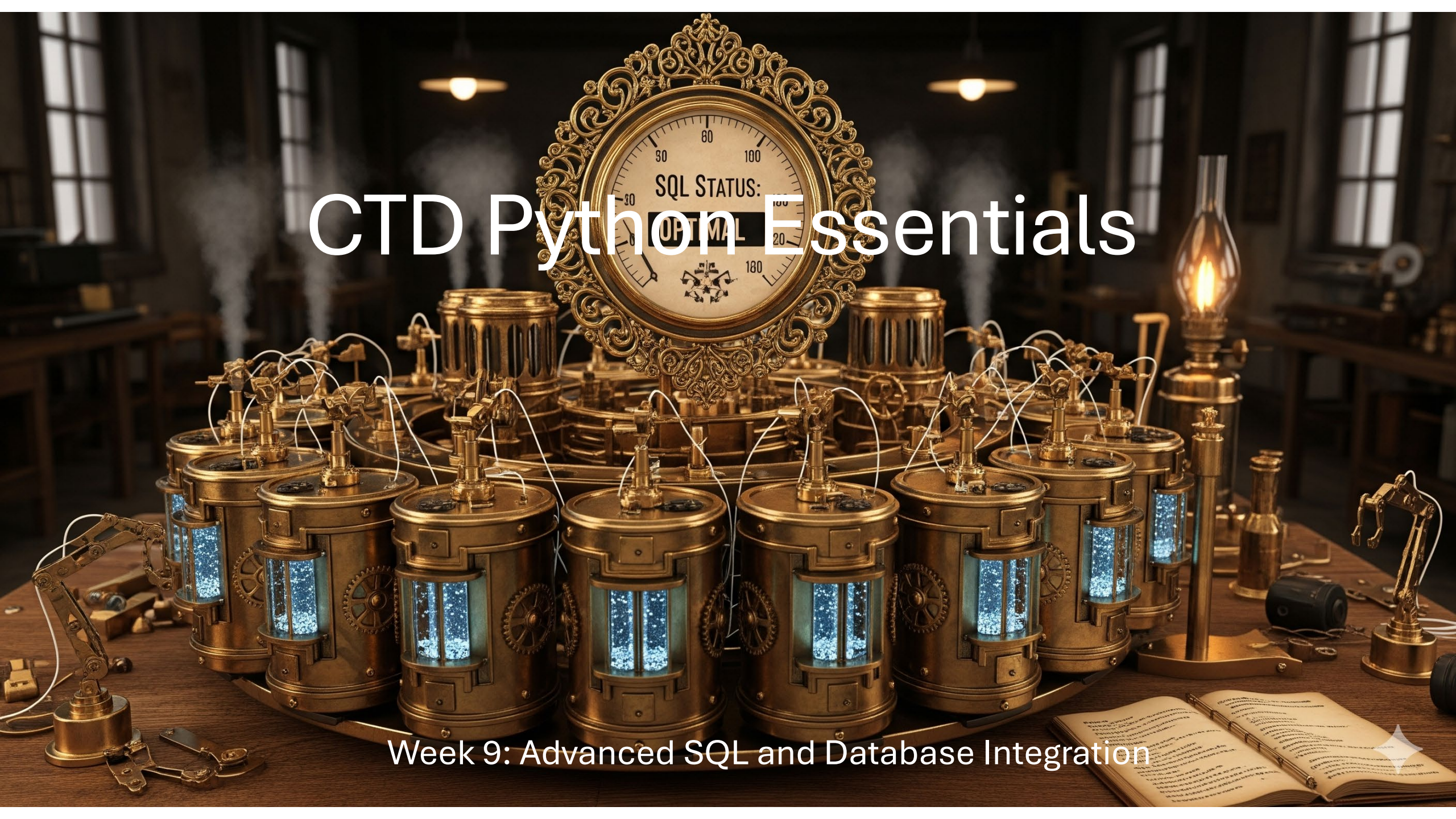


CTD Python Essentials



Week 9: Advanced SQL and Database Integration

Using Subqueries

- This github gist provides executable examples for lesson 9.
 - The code shown in the current lesson references a table which doesn't exist and has other inconsistencies.
 - [CTD Python 100 Lesson 9 Code Examples](#)
 - The code assumes a 'db' directory in your workspace
- Subqueries are nested queries which allow for intermediate operations such as calculations

```
SELECT department_id, employee_id, salary
FROM Employees AS e
WHERE salary = (
    SELECT MAX(salary)
    FROM Employees
    WHERE department_id = e.department_id
);
```

```
CREATE TABLE Projects (  
    id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    department_id INTEGER NOT NULL,  
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)  
);
```

```
SELECT (e.first_name || ' ' || e.last_name) AS employee_name,  
       p.name AS project_name  
FROM Employees AS e  
JOIN Projects AS p ON e.department_id = p.department_id  
WHERE p.name = 'Project A';
```

```
INSERT INTO Projects (name, department_id) VALUES  
( 'Project A', (SELECT department_id FROM Departments WHERE department_name = 'HR')),  
( 'Project B', (SELECT department_id FROM Departments WHERE department_name = 'Engineering')),  
( 'Project C', (SELECT department_id FROM Departments WHERE department_name = 'Finance'));
```

Complex Joins

- Retrieving data from multiple related tables
 - INNER JOIN: Retrieves records with matching values in both tables.
 - LEFT JOIN: Retrieves all records from the left table and matching records from the right.
- This example uses a JOIN to combine employee and project information through the department field
- List employees working in departments responsible for a specific project

Aggregations

- Aggregation functions like MIN(), MAX(), COUNT(), and AVG() allow you to summarize data across groups
- Calculate the minimum and maximum salaries and the number of employees in each department

```
SELECT department_id,  
       MIN(salary) AS min_salary,  
       MAX(salary) AS max_salary,  
       COUNT(employee_id) AS num_employees  
FROM Employees  
GROUP BY department_id;
```

Aggregation using HAVING

- The HAVING clause filters aggregated results after the GROUP BY operation
- List all departments where the average salary exceeds 70,000 and display the department manager
- The second version shows department_name rather than id
- Use HAVING instead of WHERE to filter aggregated results

```
SELECT department_id,  
       MIN(salary) AS min_salary,  
       MAX(salary) AS max_salary,  
       COUNT(employee_id) AS num_employees  
FROM Employees  
GROUP BY department_id;
```

```
SELECT d.department_name AS department,  
       MIN(e.salary) AS min_salary,  
       MAX(e.salary) AS max_salary,  
       COUNT(e.employee_id) AS num_employees  
FROM Employees AS e  
JOIN Departments AS d  
  ON e.department_id = d.department_id  
GROUP BY d.department_id, d.department_name  
ORDER BY d.department_name;
```

Using an index for better performance

- For large tables and index can be created to speed up queries
 - This example will speed up queries which filter by department

```
CREATE INDEX idx_department ON Employees(department_id);
```

Transactions

- SQL uses transactions to ensure consistency
- Avoids race conditions and avoids corruption on error conditions
- Transactions can be committed or rolled back on an exception
 - The [gist](#) contains an extended example showing the value of rollbacks

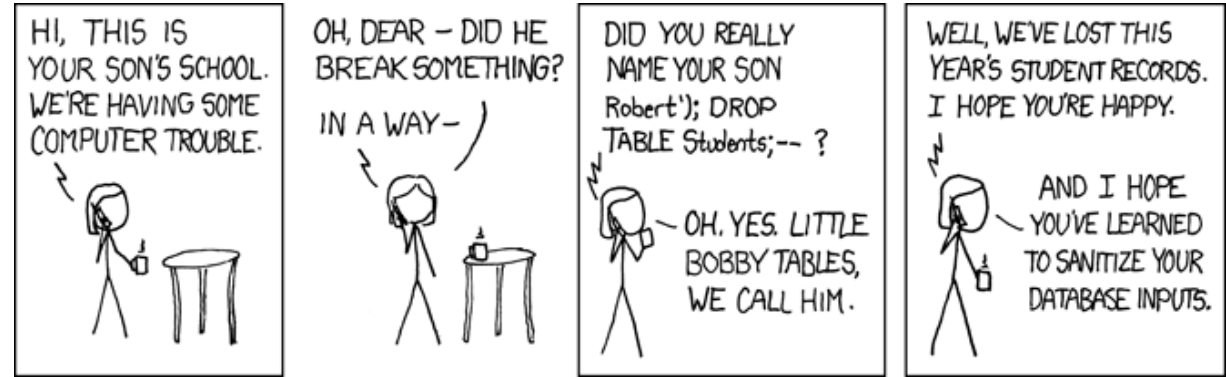
```
conn = sqlite3.connect("company.db")
cursor = conn.cursor()

try:
    cursor.execute("INSERT INTO Employees (name, department_id) VALUES ('John Doe', 2)")
    cursor.execute("INSERT INTO Employees (name, department_id) VALUES ('Jane Smith', 3)")
    conn.commit() # Commit transaction
except Exception as e:
    conn.rollback() # Rollback transaction if there's an error
    print("Error:", e)

conn.close()
```

SQL Injection

- A reminder about injection attacks
- Never allow user inputs to be run as SQL code
- Always use parameters, these are checked for potential attacks
- Never use interpolation



```
cursor.execute("SELECT * FROM Employees WHERE department_id = ?", (department_id,))
```

NOT

```
cursor.execute(f"SELECT * FROM Employees WHERE department_id = {department_id};")  
# or, equally bad:  
cursor.execute("SELECT * FROM Employees WHERE department_id = " + department_id + ";")
```

Window Functions

- Window functions operate on a specific range of rows
- calculating the rank of employees within a department based on salary

```
SELECT (e.first_name || ' ' || e.last_name) AS employee_name,  
       e.salary,  
       e.department_id,  
       RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS rank  
FROM Employees AS e;
```

Date and Time Functions

- Sqlite doesn't have a date or time column type
- You can still use the SQL JULIANDATE operation
- All SQLite date/time functions (date(), time(), datetime(), julianday(), strftime()) take a time string
- Common time string forms:
 - 'YYYY-MM-DD'
 - 'YYYY-MM-DD HH:MM'
 - 'YYYY-MM-DD HH:MM:SS'
 - ISO-8601 with T: 'YYYY-MM-DDTHH:MM:SS'
 - now

```
SELECT (e.first_name || ' ' || e.last_name) AS employee_name,  
       e.hire_date,  
       JULIANDAY('now') - JULIANDAY(e.hire_date) AS tenure_in_days  
FROM Employees AS e;
```

Development and Debugging

- Use the sqlite command line to test sub-queries and full queries before running in a python script
 - Verify trusted sub-operations
- Run python code snippets in Jupyter on vscode of Kaggle to test individual operation
- Use exceptions (try...except) to catch error conditions
 - Create custom exceptions for specific conditions
 - sqlite3.IntegrityError – catches things like duplicates
 - sqlite3.OperationalError – catches illegal sql, incorrect table...

```
def add_callsign(cursor, df_row_dict):  
    "Add a single callsign to the database based on a row dict"  
    try:  
        columns = ', '.join(df_row_dict.keys())  
        placeholders = ', '.join(['?'] * len(df_row_dict))  
        sql = f"INSERT INTO callsign ({columns}) VALUES ({placeholders})"  
        cursor.execute(sql, tuple(df_row_dict.values()))  
        return cursor.lastrowid  
    except sqlite3.IntegrityError:  
        print(f"{df_row_dict['callsign']} is already in the database.")
```

Q&A and Demo

Gist with code examples:

[CTD Python 100 Lesson 9 Code Examples](#)

